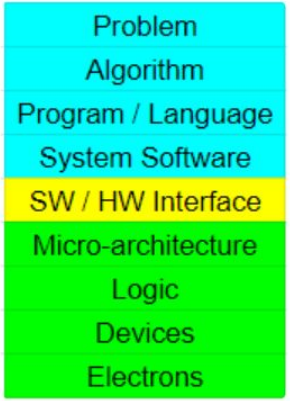
Lecture 2: Computer architecture review & Systems programming

CS211 - Operating systems

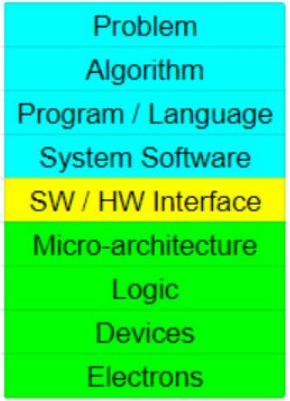
Spring 2026

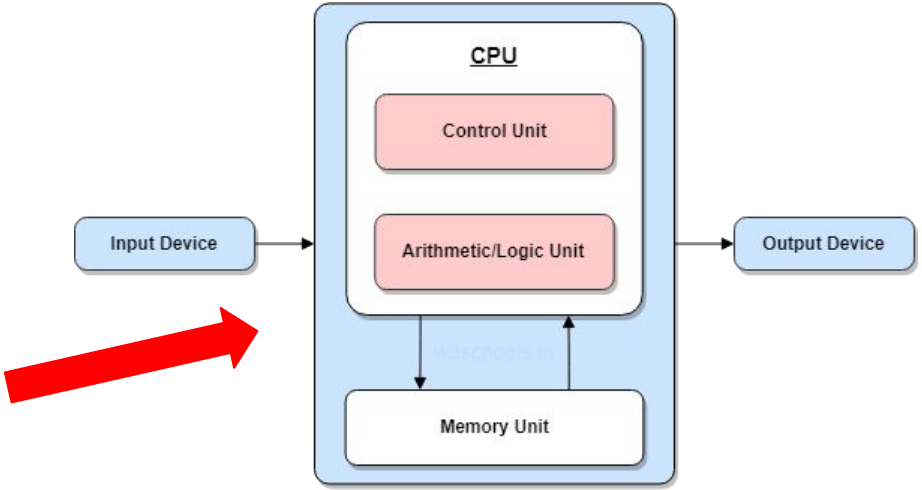
Computer architecture overview

Computer architecture overview

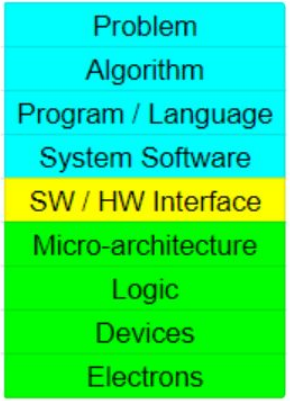


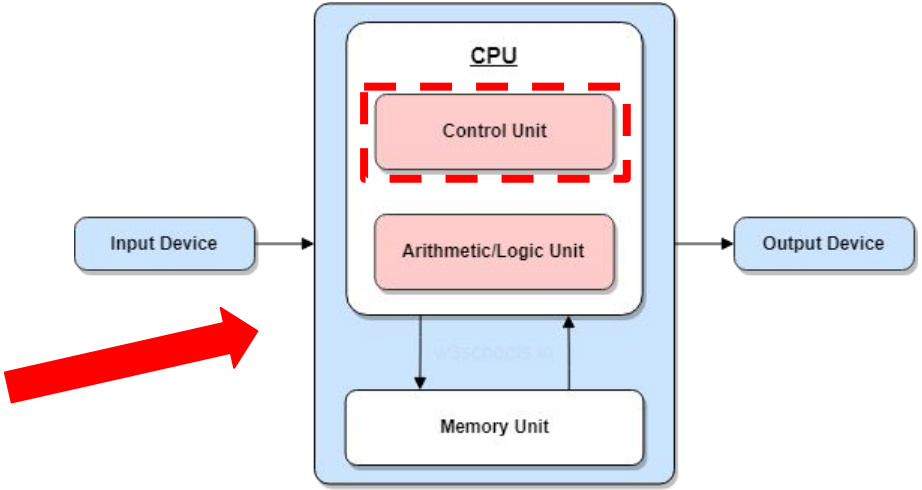
Computer architecture overview



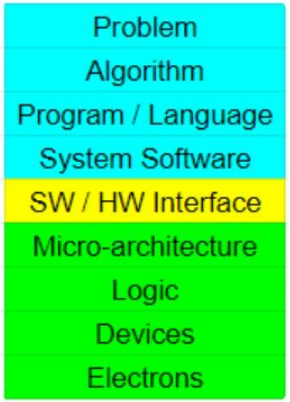


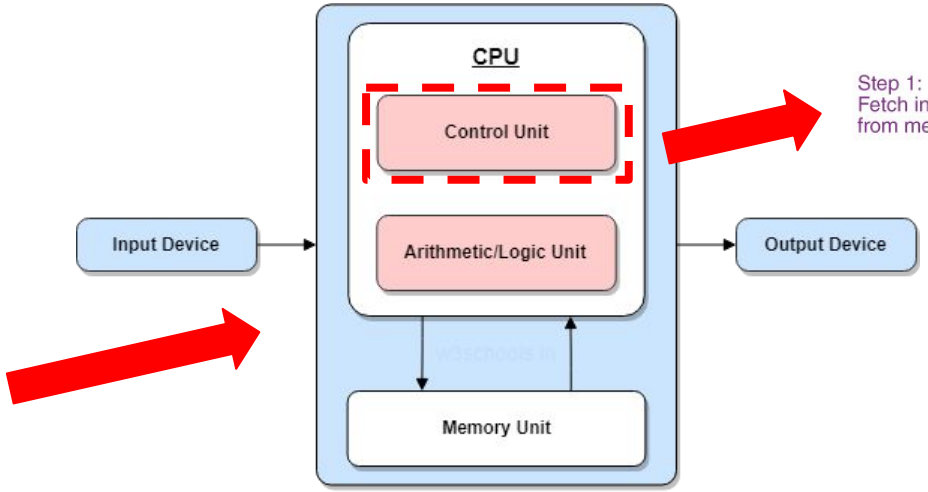
Computer architecture overview

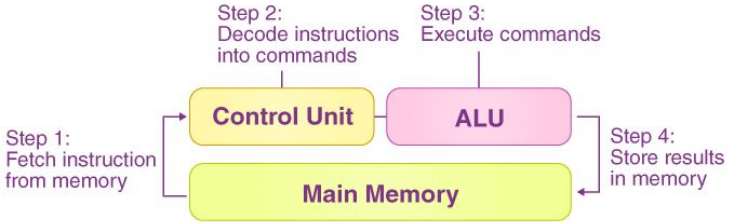




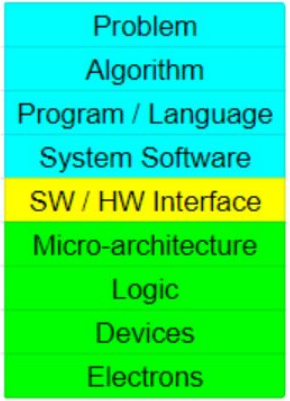
Computer architecture overview

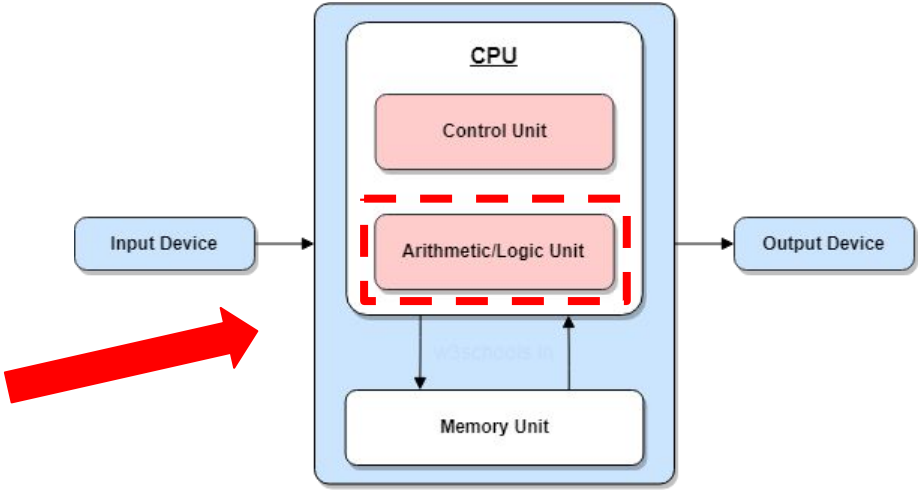




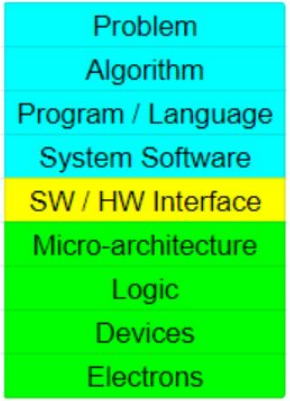


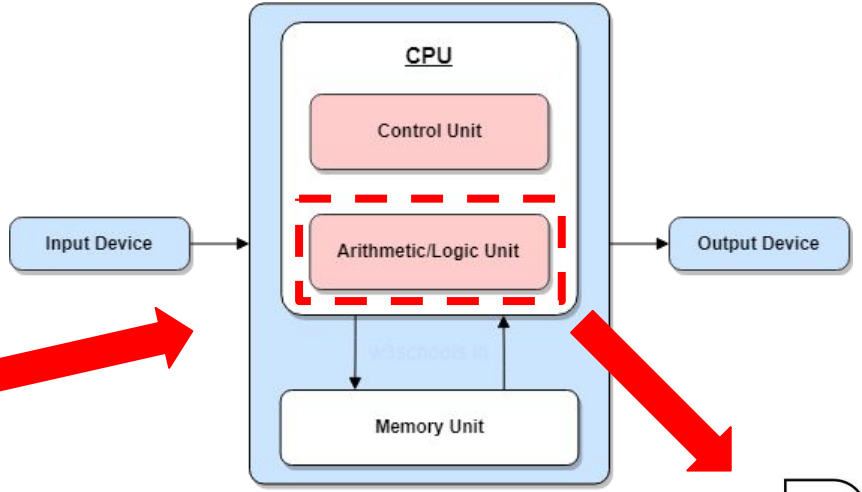
Computer architecture overview

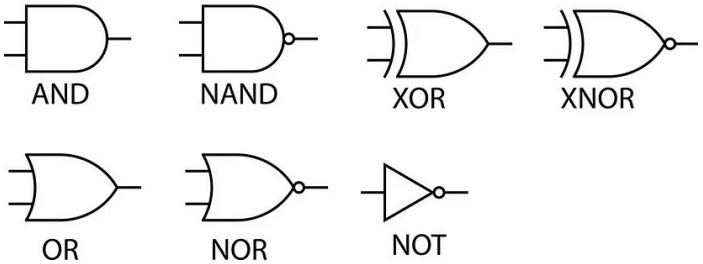




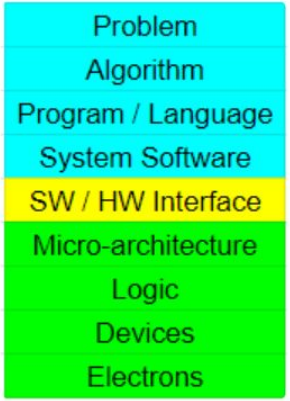
Computer architecture overview

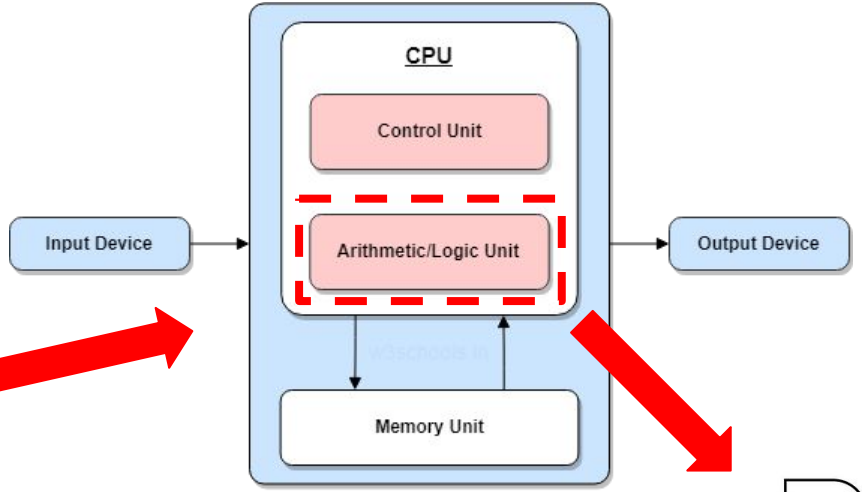


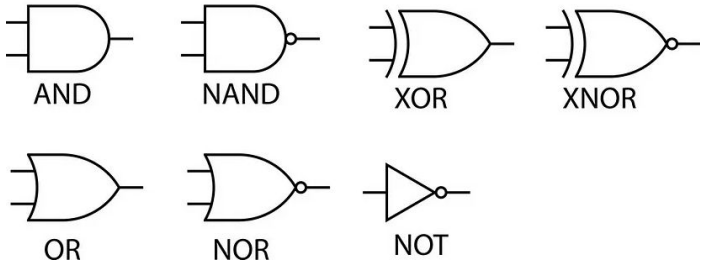


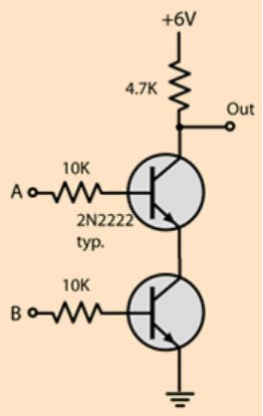


Computer architecture overview

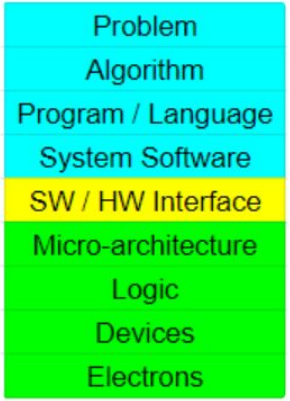


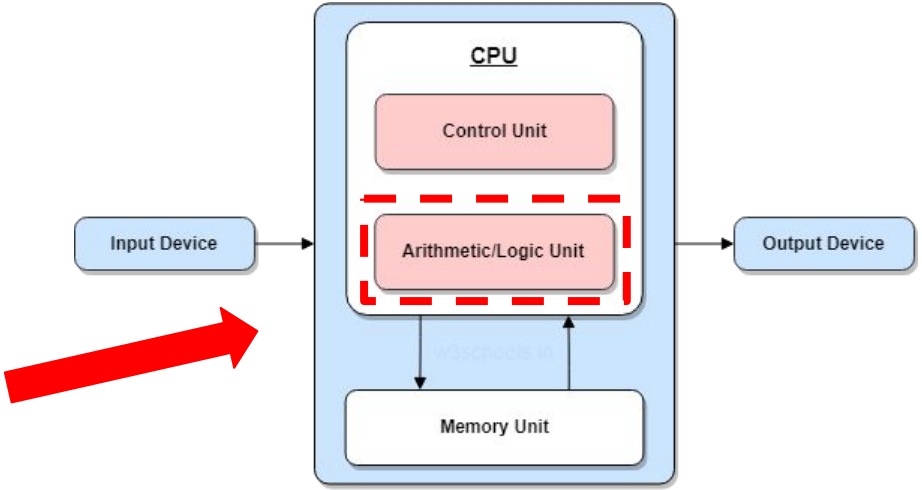




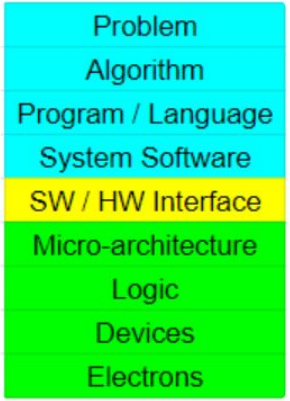


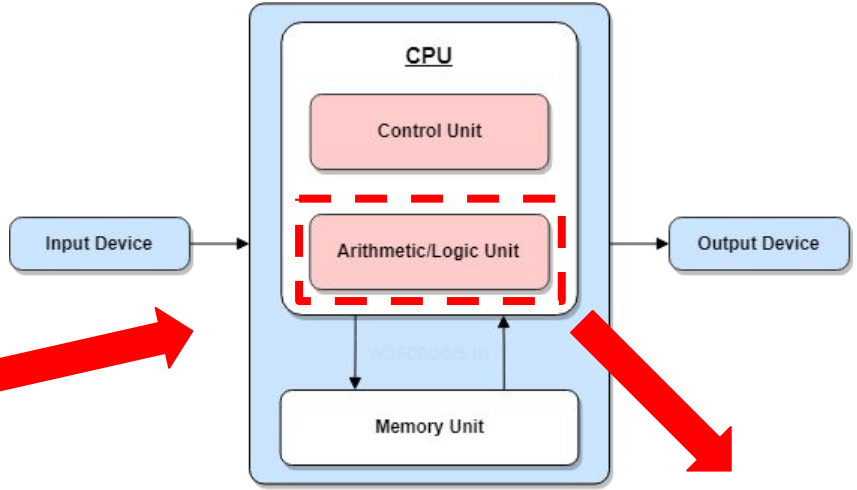
Computer architecture overview

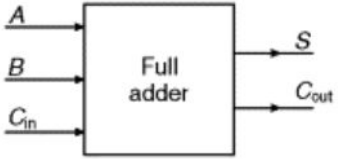




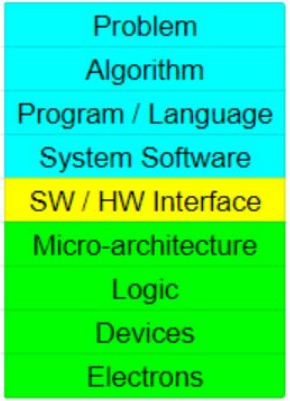
Computer architecture overview

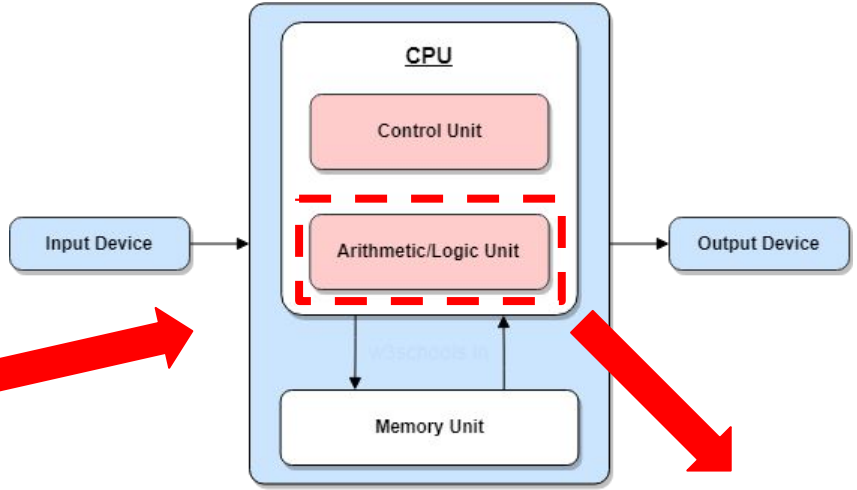


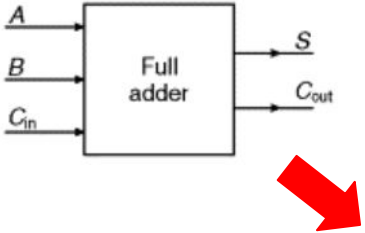




Computer architecture overview

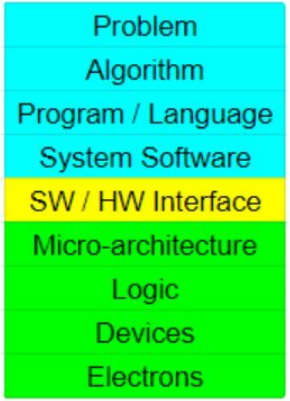


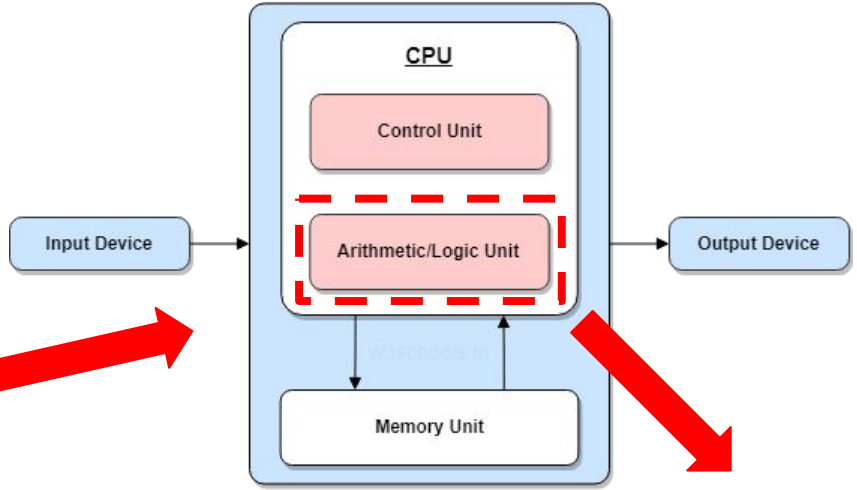


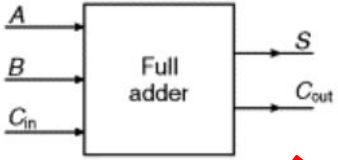


A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

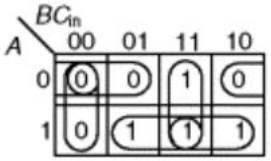
Computer architecture overview



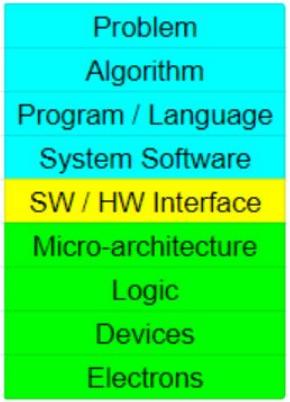


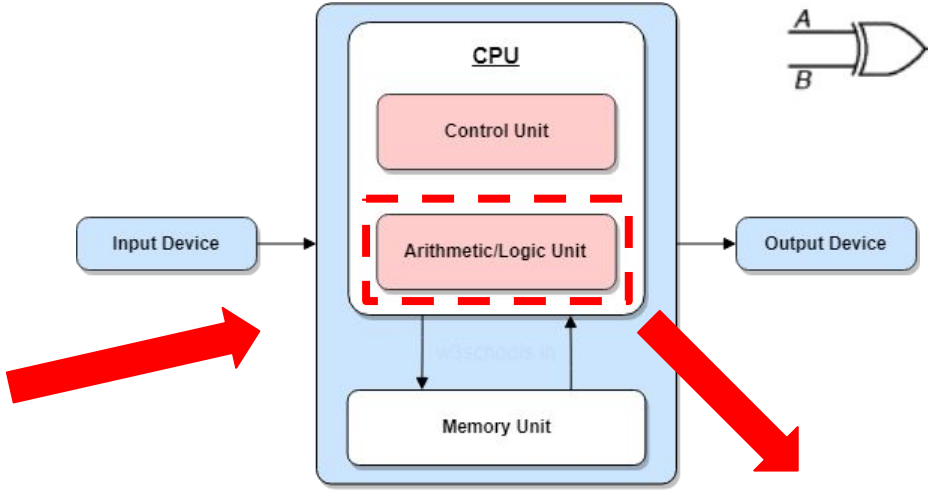


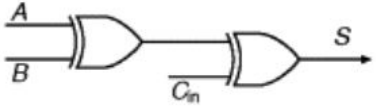
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

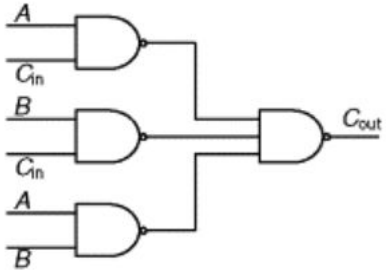


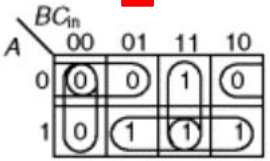
Computer architecture overview

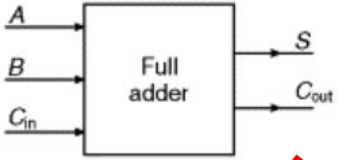






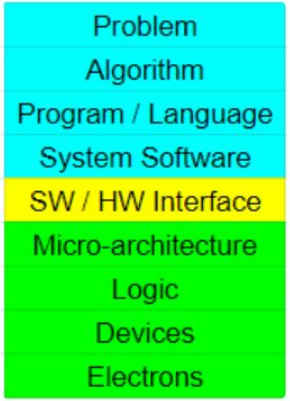


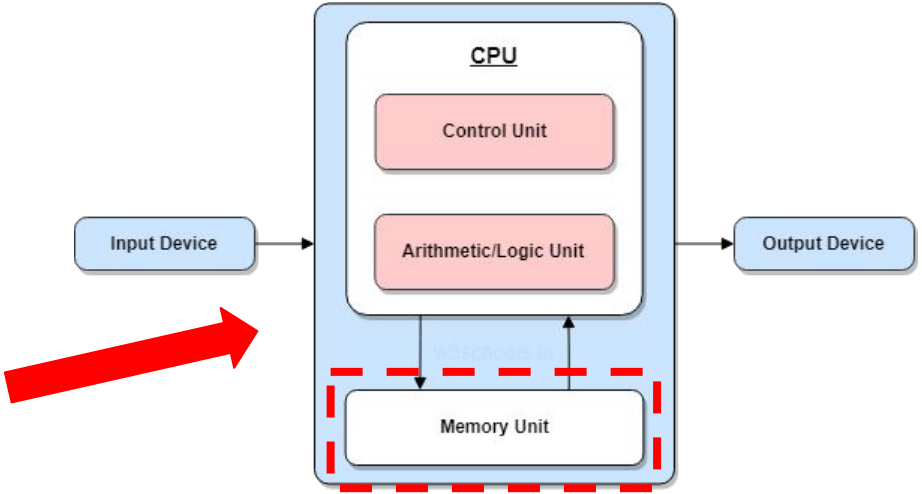




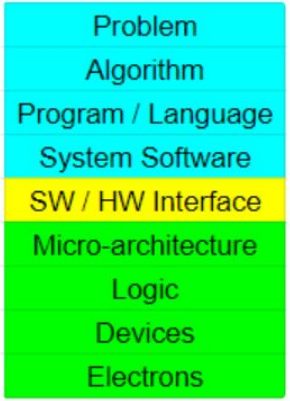
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

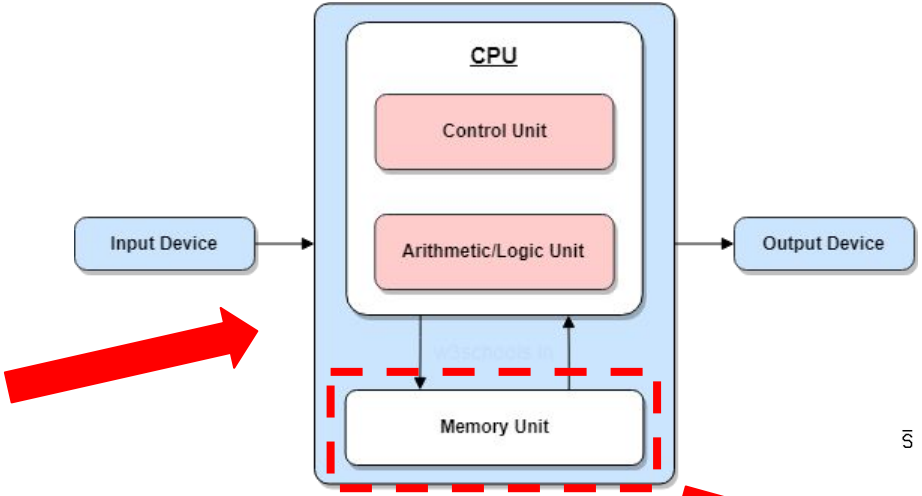
Computer architecture overview

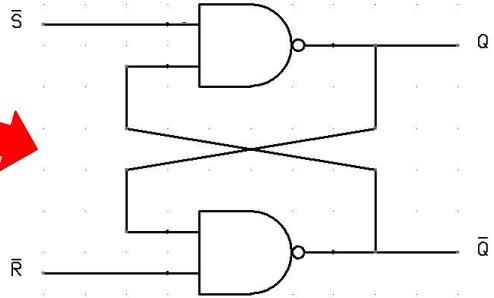




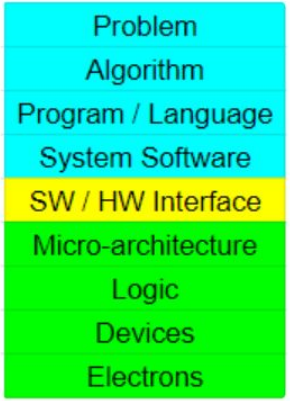
Computer architecture overview

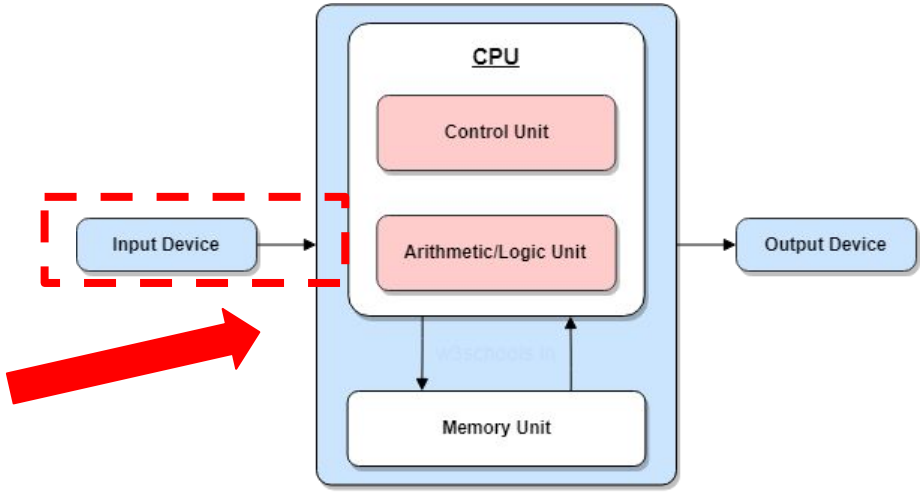




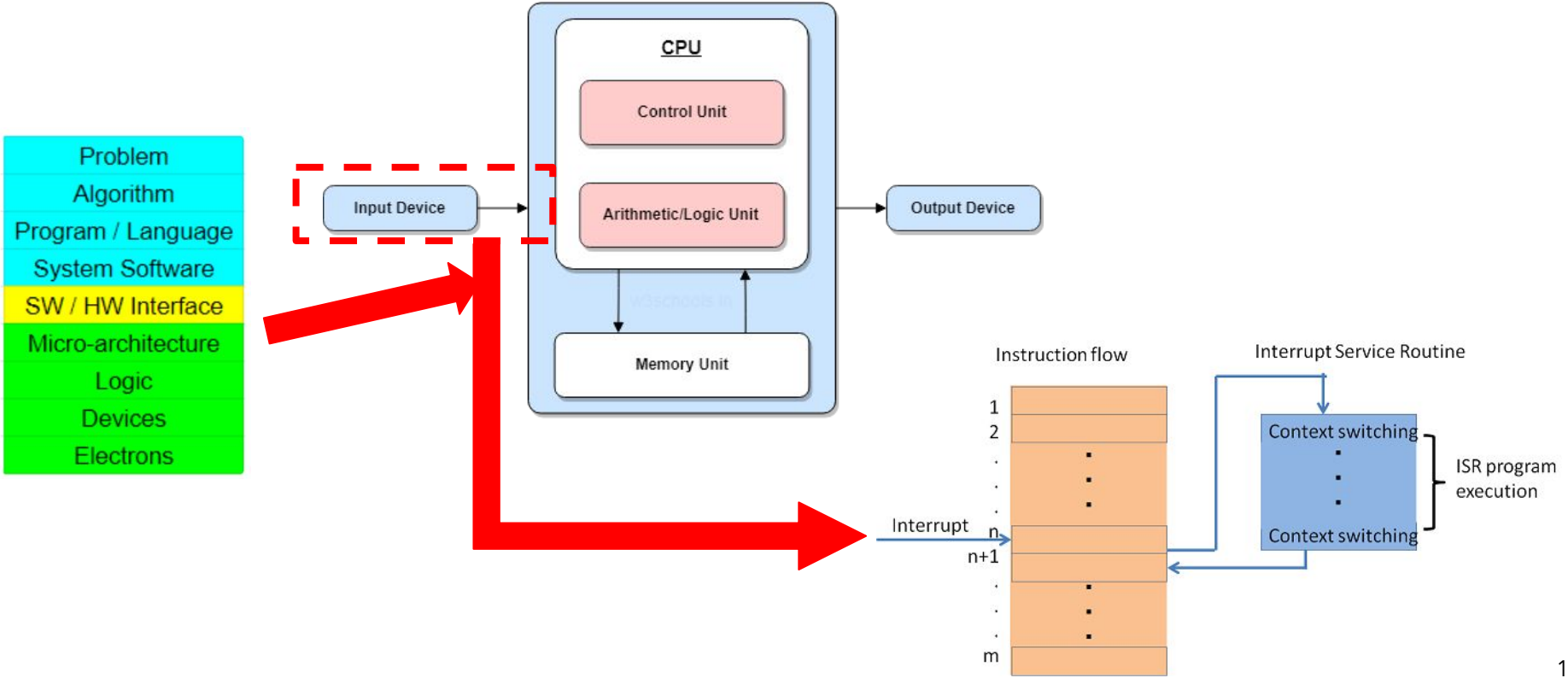


Computer architecture overview

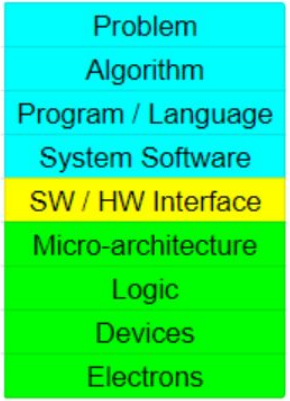


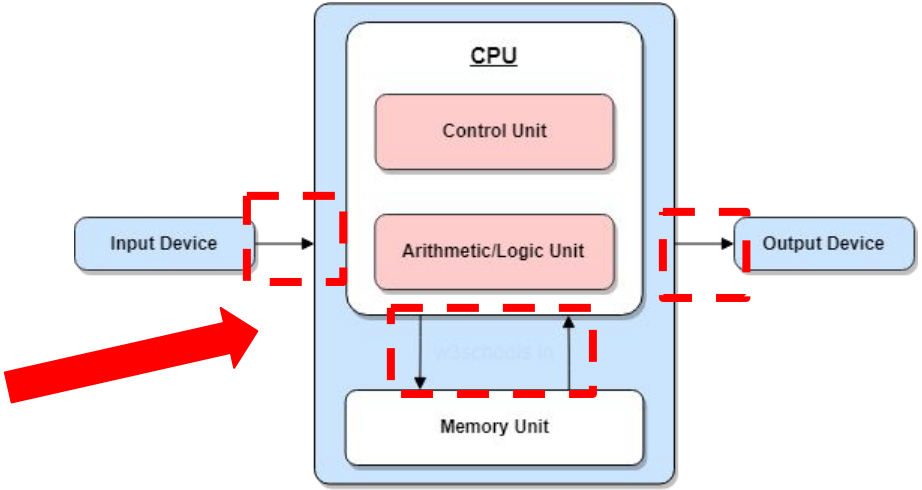


Computer architecture overview

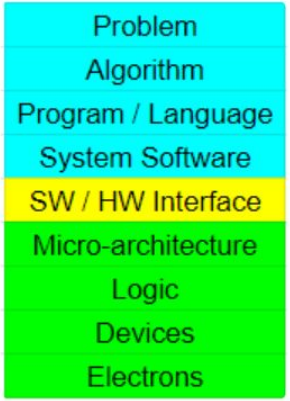


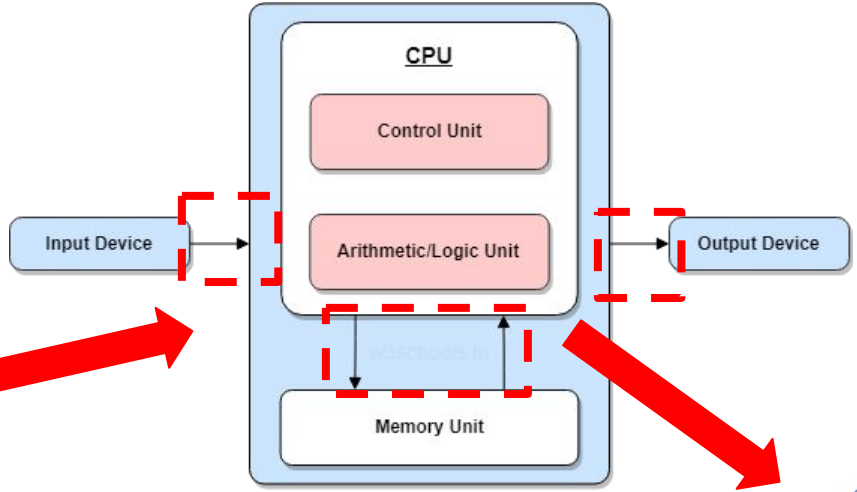
Computer architecture overview

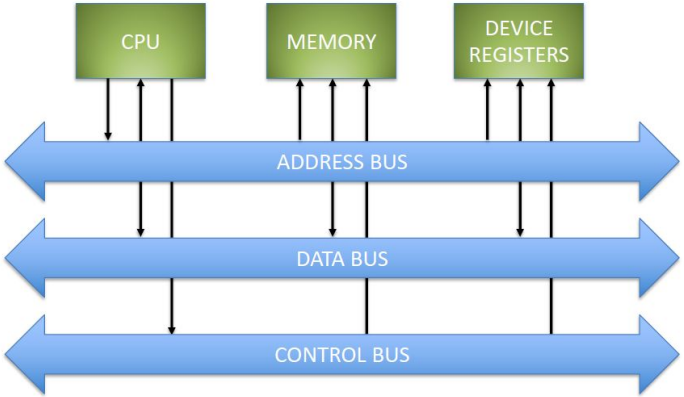




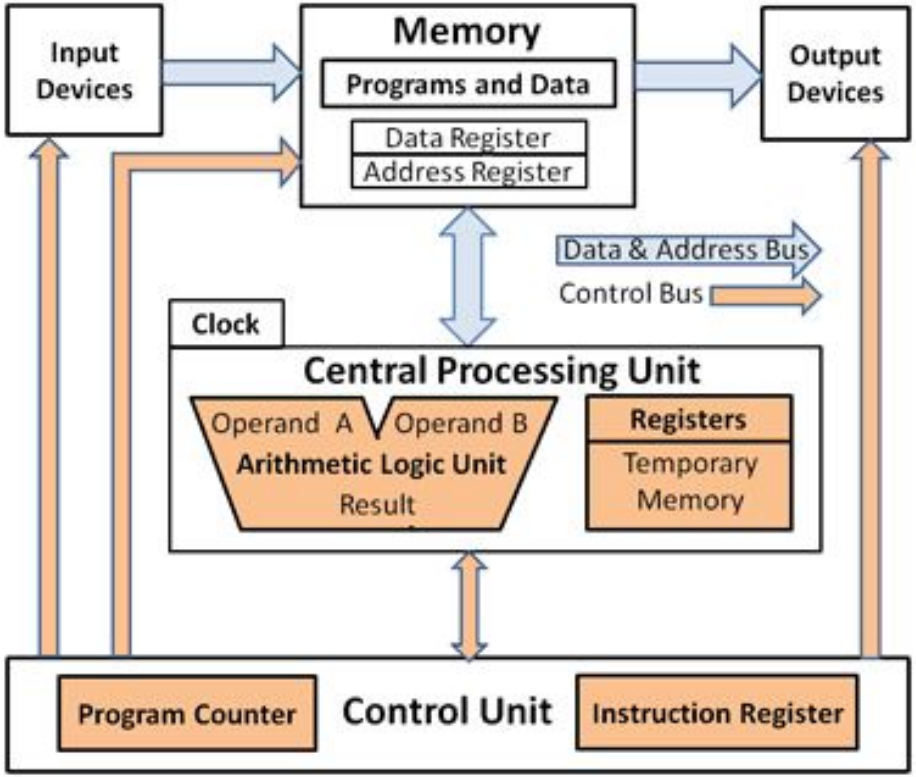
Computer architecture overview







Von Neumann Architecture



Instruction set architecture (ISA)

- Complex instruction set computer (CISC)
  - x86
- Reduced instruction set computer (RISC)
  - RISC-V
  - ARM
  - PowerPC

Booting an OS?

1. CPU starts from a predefined location in the firmware (BIOS or UEFI)
   - in supervisor mode with interrupts disabled
2. Loads “boot loader” code from specified storage or network device into memory and runs it
3. Boot loader loads O.S. kernel code into memory and runs it
   - Determine location/size of physical memory
   - Set up initial MMU/page tables
   - Initialize the interrupt vector
   - Determine which devices the computer has
   - Invoke device driver initialization code for each
   - Initialize file system code
   - Load first process from file system
   - Start first process

Assembly language

- https://godbolt.org/
- https://cs61.seas.harvard.edu/site/2019/Asm/#Registers
- Basic instructions
  - Accessing data
    - mov, push, pop
      - mov src, dst
  - Manipulating data
    - add, sub, mul, div, and, or, not, …
  - Control flow
    - jmp, je, jle, jg, jge, …
    - call, ret
    - int

C/C++ language

- Compile and run a program
- Basic syntax
- Data types
  - Pointers
  - Arrays & strings
    - Malloc & free
  - Structs

Compile and run a C program

gcc your_program.c -o your_program

./intro your_program argument_1 argument_2 …

gcc -S your_program.c

Basic syntax

- Similar to Java
  - printf

Pointers

```
int x = 1;
int* p; //pointer
&x = 1; // take the address
*p = 1; // update the value
x = p + 1;
&x = p;
p = &x;
```

Arrays & strings

```
int NUM_ELEMS = 3;
int arr1[NUM_ELEMS];
for (int i = 0; i < NUM_ELEMS; i++) {
    arr1[i] = i;
}
int *arr2 = malloc(sizeof(int) * NUM_ELEMS);
```

Malloc & free

```
ptr = (int*) malloc(100 * sizeof(int));

free(ptr);
```

Structs

```
struct coord {
    int x;
    int y;
};
int main(int argc, char *argv[]) {
    struct coord c1;
    c1.x = 3;
    c1.y = 4;

    struct coord *c2 = malloc(sizeof(struct coord));
    (*c2).x = 3;
    c2->y = 4;

    printf("x: %d, y: %d\n", c1.x, c1.y);
    printf("x: %d, y: %d\n", c2->x, c2->y);

    return 0;
}
```

Summary

- Computer architecture
  - Von Neumann Architecture
    - CPU
    - Memory
    - I/O devices
  - Instruction set architecture
    - x86, RISC-V, ARM
- Systems programming
  - Assembly, C/C++